TREAP

25125039 - Nguyễn Lê Trân
25125065 - Bùi Viết Thành
25125071 - Lê Quỳnh Anh
25125078 - Đinh Nguyễn Hồng Ánh

@2023, University of Science HCMC - Advanced Program in Computer Science

Definition and History

TREAP = TREE + HEAP

Definition

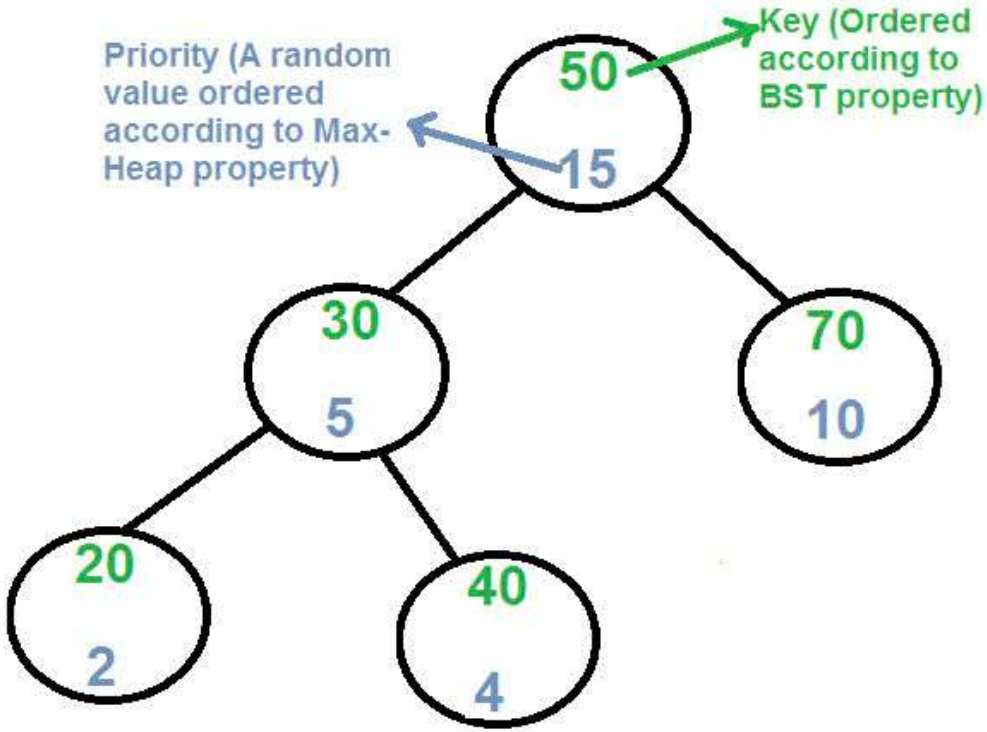
A treap is a randomized binary search tree in which each node stores two values in the set of (X, Y):

(1) a **key (X value)**, used to maintain the **binary-search-tree (BST)** ordering

(2) a **priority (Y value)**, chosen randomly at insertion time and used to maintain a **heap** ordering.

Copyright © University of Science, VNUHCM - APCS 2023

Definition



Copyright © University of Science, VNUHCM - APCS 2023

Historical notes

Treaps were introduced by Seidel and Aragon (1989) under the name **randomized search trees.**

Copyright © University of Science, VNUHCM - APCS 2023

Balanced Search Tree (AVL, Red-Black) vs. Treap in their performances to achieve balance?

Copyright © University of Science, VNUHCM - APCS 2023

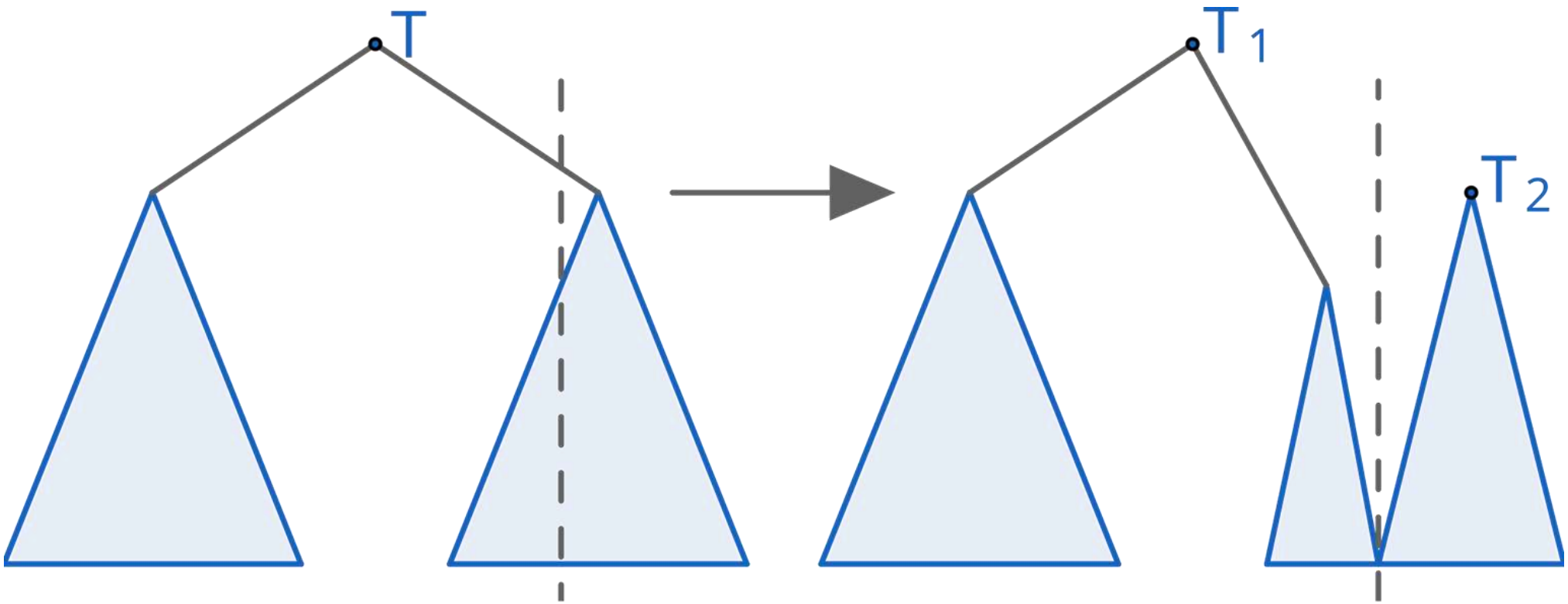
Balanced Search Tree	Treap
Achieve balance by maintaining **strict structural constraints** (balance factor and coloring rules)	Achieve balance in expectation through **randomization**

Copyright © University of Science, VNUHCM - APCS 2023

Core Data Structure

@2023, University of Science HCMC - Advanced Program in Computer Science

Split



Copyright © University of Science, VNUHCM - APCS 2023

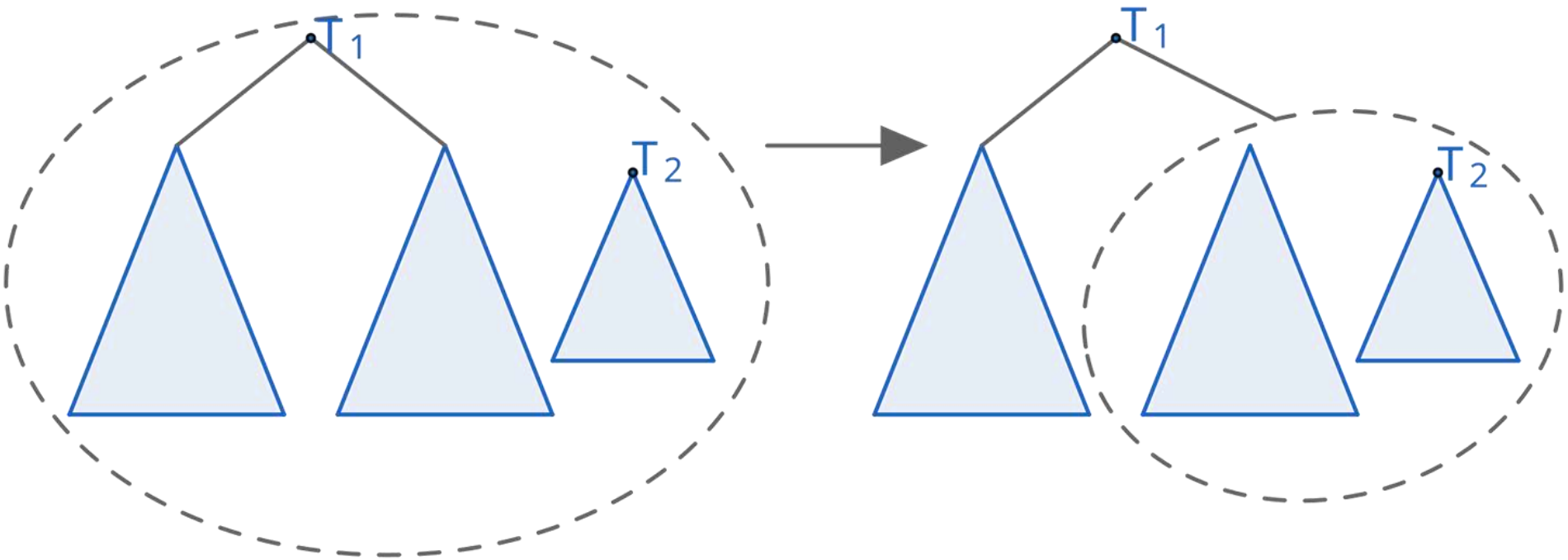
Split – pseudocode

Algorithm 1 Split(T, x) (BST by key; heap by priority)

```
procedure SPLIT(T, x)
    if T = null then
        return (null, null)
    if T.key ≤ x then
        (A, B) ← SPLIT(T.right, x)
        T.right ← A
        return (T, B)
    else
        (A, B) ← SPLIT(T.left, x)
        T.left ← B
        return (A, T)
```

Copyright © University of Science, VNUHCM - APCS 2023

Merge



Copyright © University of Science, VNUHCM - APCS 2023

Merge – pseudocode

Algorithm 2 Merge(L, R) where $\max(L.key) \leq \min(R.key)$

```
procedure MERGE(L, R)
    if L = null then
        return R
    if R = null then
        return L
    if L.prio > R.prio then                    ▷ max-heap example
        L.right ← MERGE(L.right, R)
        return L
    else
        R.left ← MERGE(L, R.left)
        return R
```

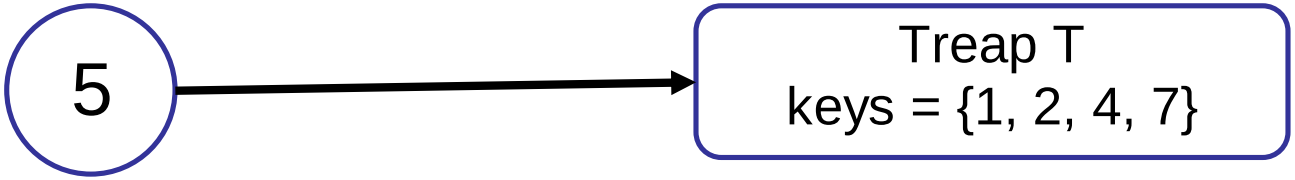
Copyright © University of Science, VNUHCM - APCS 2023

Insertion

Treap T
keys = {1, 2, 4, 7}

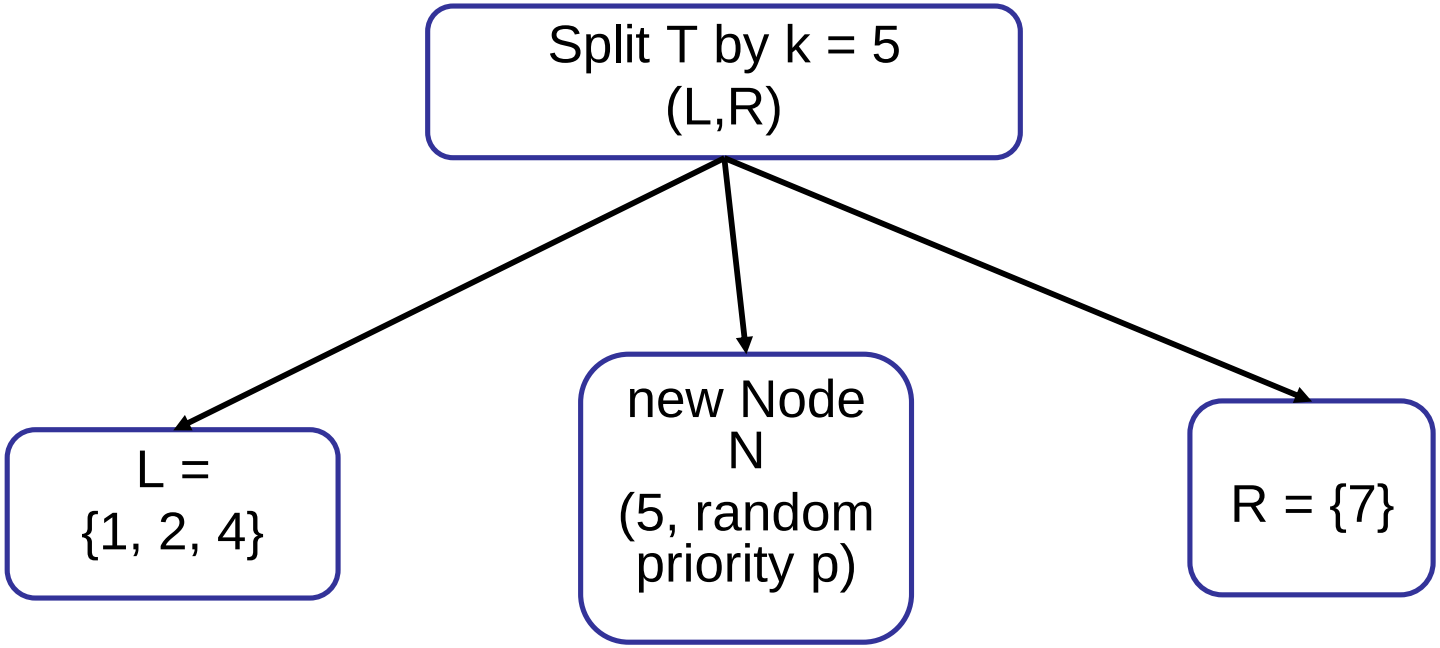
Copyright © University of Science, VNUHCM - APCS 2023

Insertion



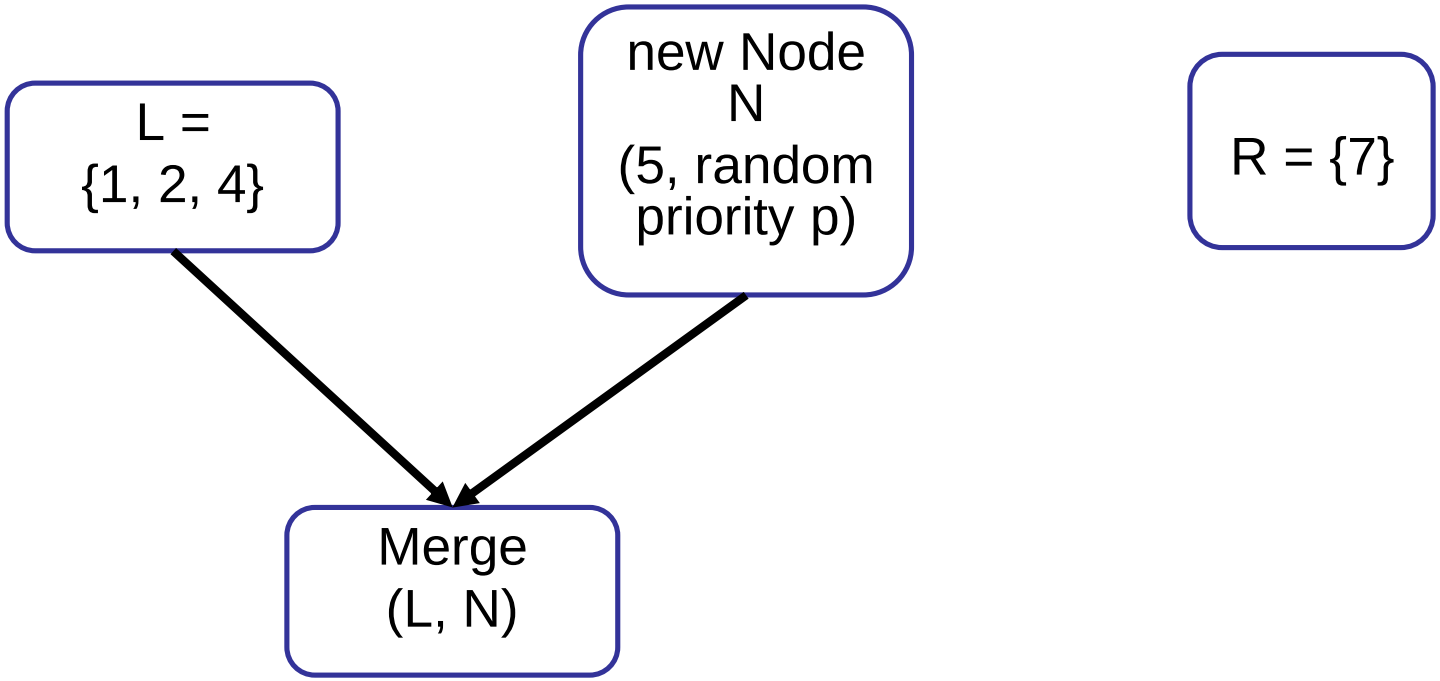
Copyright © University of Science, VNUHCM - APCS 2023

Insertion



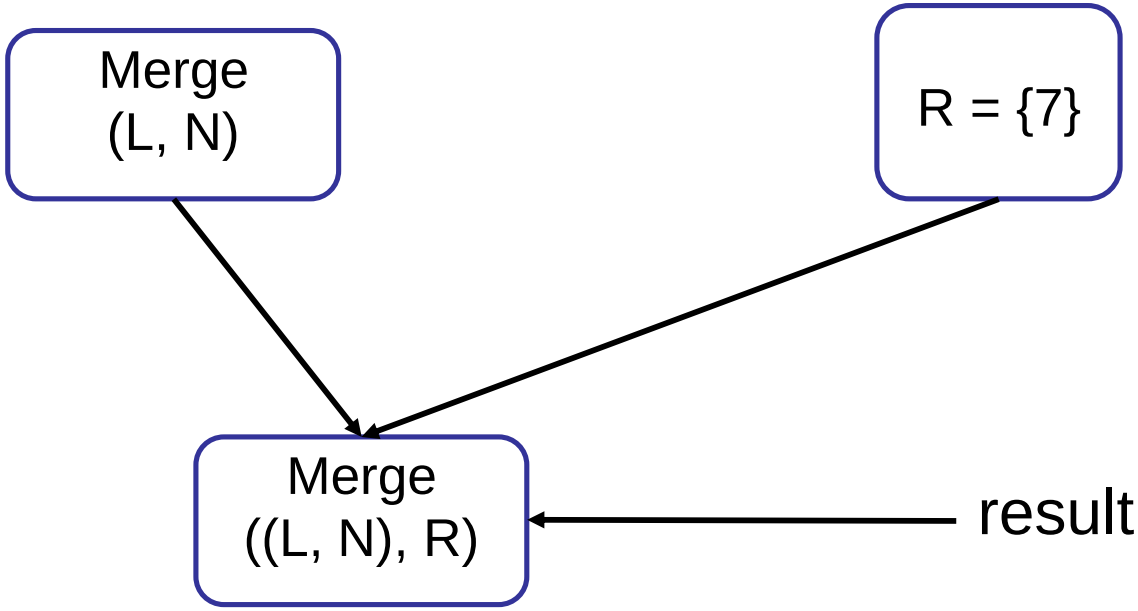
Copyright © University of Science, VNUHCM - APCS 2023

Insertion



Copyright © University of Science, VNUHCM - APCS 2023

Insertion



Copyright © University of Science, VNUHCM - APCS 2023

Deletion

Deletion a key k follows the reverse steps of insertion, where:

- Locate a node with key *k* using BST property
- Remove it by merging its subtrees:

$$T \leftarrow \text{Merge}(T.left,\ T.right)$$

Copyright © University of Science, VNUHCM - APCS 2023

Searching

Searching is identical to a standard BST search: compare keys and move left/right until a match is found or a null pointer is reached.

Copyright © University of Science, VNUHCM - APCS 2023

Implicit treaps

Instead of storing keys, we use position as the key.

This allows us to:

- treat it like an array
- move subarrays efficiently
- support range operations like reverse

This is very useful in competitive programming

Copyright © University of Science, VNUHCM - APCS 2023

Time complexity

Table 1: Typical complexity summary for treap operations (expected vs. worst-case).

Operation	**Expected**	**Worst-case**
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Space	$O(n)$	$O(n)$

Copyright © University of Science, VNUHCM - APCS 2023

Applications

Ordered set / map (dictionary)

- Supports: Insert, delete, search
- Expected **O(log n)** time
- Comparable to: AVL / Red-Black Trees
- Key advantage:
  Much simpler to implement
- Practical benefit:
  Less code → easier optimization

Copyright © University of Science, VNUHCM - APCS 2023

Order statistics and range queries (augmented treaps)

- Augment nodes with **subtree size / aggregates**
- Supports: k-th smallest element, rank of a key, count in range
- Can maintain **sum / min / max over ranges**
- Update info in **O(1) per operation**
- Useful for **dynamic query problems**

Copyright © University of Science, VNUHCM - APCS 2023

Dynamic sequence editor (implicit treap / rope)

- Implicit treaps use the in-order position as the key, representing a **sequence instead of a set**.

- With **lazy propagation**, range updates (e.g., reversing a segment) can be handled quickly.

- **dynamic sequence manipulation** in algorithmic problems.

Copyright © University of Science, VNUHCM - APCS 2023

Flexible Structure for Custom Problems

- Useful when:

Standard libraries are not enough

- Can combine:

Ordering + custom queries + modifications

- Key strength:

Write your own structure easily

Copyright © University of Science, VNUHCM - APCS 2023

Quizzes

Q1. In a **treap**, each node stores a pair **(X, Y)**. Which statement is correct?

A. The tree satisfies the BST property by Y values and heap property by X.
B. The tree satisfies the BST property by X values and heap property by Y.
C. Both properties are based on X values.
D. Both properties are based on Y values.

Copyright © University of Science, VNUHCM - APCS 2023

Q1. In a **treap**, each node stores a pair **(X, Y)**. Which statement is correct?

A. The tree satisfies the BST property by Y values and heap property by X.

B. The tree satisfies the BST property by X values and heap property by Y.

C. Both properties are based on X values.

D. Both properties are based on Y values.

Copyright © University of Science, VNUHCM - APCS 2023

Q2. What is the purpose of using a **random priority Y value** in a treap?

A. To reduce memory usage.
B. To guarantee perfect balance.
C. To avoid degeneration and achieve expected O(logN) time complexity.
D. To make search operations O(1).

Copyright © University of Science, VNUHCM - APCS 2023

Q2. What is the purpose of using a **random priority Y value** in a treap?

A. To reduce memory usage.

B. To guarantee perfect balance.

C. To avoid degeneration and achieve expected O(logN) time complexity.

D. To make search operations O(1).

Copyright © University of Science, VNUHCM - APCS 2023

Q3. What is the expected time complexity for each of **Insert**, **Search**, and **Erase** operation in a treap?

A. O(logN).
B. O(1).
C. O(N).
D. O(NlogN).

Copyright © University of Science, VNUHCM - APCS 2023

Q3. What is the expected time complexity for each of **Insert**, **Search**, and **Erase** operation in a treap?

A. O(logN).
B. O(1).
C. O(N).
D. O(NlogN).

Copyright © University of Science, VNUHCM - APCS 2023

Q4. What does the operation **Split(T, X)** do?

A. Splits the treap into two trees based on priority

B. Splits the treap into two trees: left tree: keys ≤ X & right tree: keys > X

C. Deletes all nodes with key > X

D. Splits the tree based on height

Copyright © University of Science, VNUHCM - APCS 2023

Q4. What does the operation **Split(T, X)** do?

A. Splits the treap into two trees based on priority

B. Splits the treap into two trees: left tree: keys ≤ X & right tree: keys > X

C. Deletes all nodes with key > X

D. Splits the tree based on height

Copyright © University of Science, VNUHCM - APCS 2023

Q5. What condition is required for the **Merge(T1, T2)** operation?

A. All priorities in T1 must be greater than those in T2
B. Both trees must have the same number of nodes
C. All keys in T1 must be less than all keys in T2
D. Both trees must be balanced

Copyright © University of Science, VNUHCM - APCS 2023

Q5. What condition is required for the **Merge(T1, T2)** operation?

A. All priorities in T1 must be greater than those in T2

B. Both trees must have the same number of nodes

C. All keys in T1 must be less than all keys in T2

D. Both trees must be balanced

Copyright © University of Science, VNUHCM - APCS 2023

Datasets

1. Synthetic data

Queries:

- 50% Find
- 40% Insert
- 10% Delete

Data:

- Totally random data to compare with Arrays
- Data with an increasing trend to compare with BST

Copyright © University of Science, VNUHCM - APCS 2023

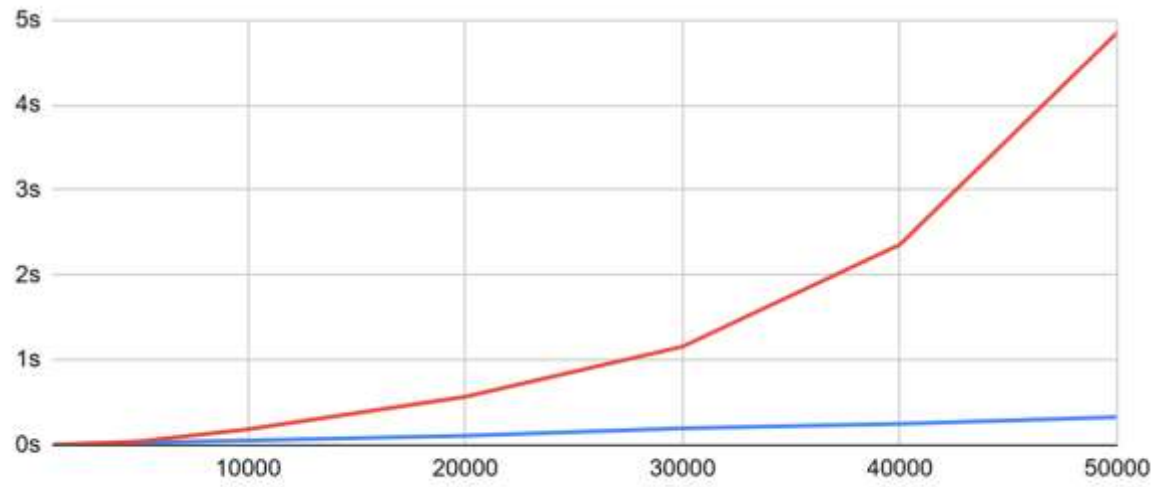
1. Synthetic data
2. Google Web Traffic Time Series

Real-world daily traffic data for 144k Wikipedia articles recorded from 1/7/2015 to 31/12/2015

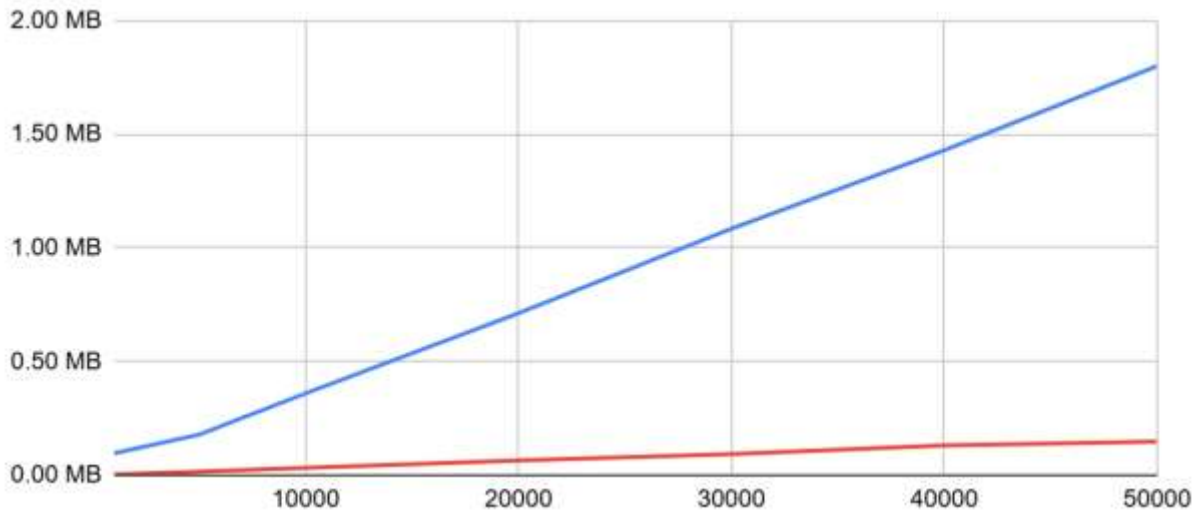
Problem: Continuously maintaining and querying the median value as new data points arrive in the stream.

Copyright © University of Science, VNUHCM - APCS 2023

Comparisons between using Treap and array for find, insert, delete queries



Runtime

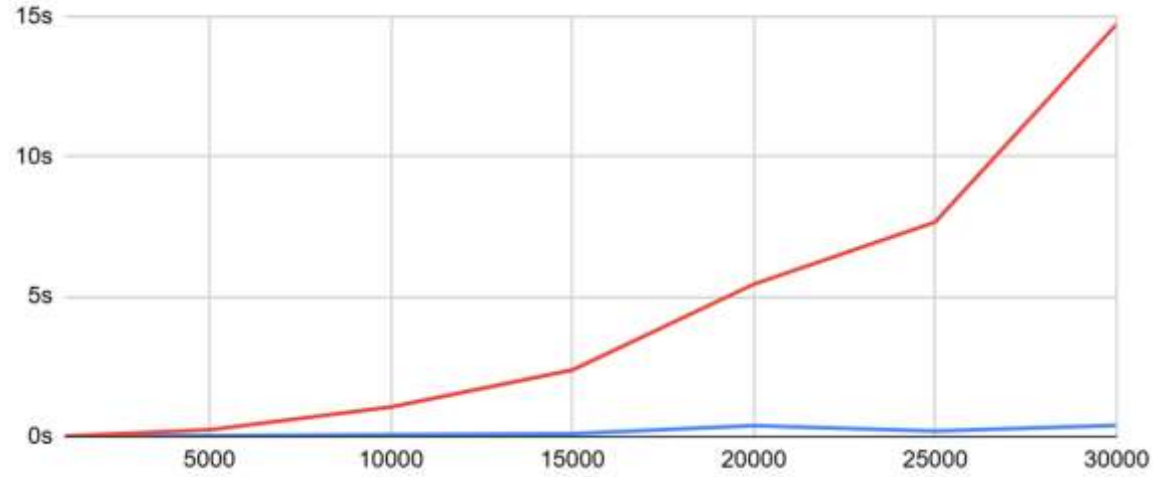


Memory Usage

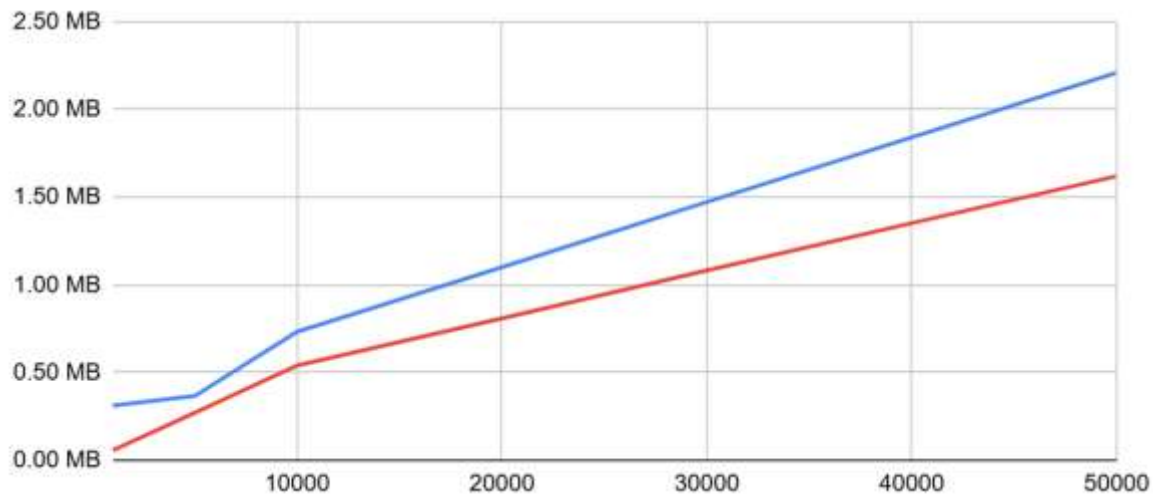
Copyright © University of Science, VNUHCM - APCS 2023

Comparisons between using Treap and BST in a dataset with an increasing trend

Treap BST



Runtime



Memory Usage

Copyright © University of Science, VNUHCM - APCS 2023

Assignment

Problem: ORDERSET - Order statistic set

You need to maintain a **dynamic set of integers** supporting the following operations:

- `I` x → Insert x into the set (only if not already present)
- `D` x → Delete x from the set (if present)
- `K` k → Find the **k-th smallest element**
- `C` x → Count how many elements are **strictly smaller than x**

💡 Hint:

- For each node, store the size of its subtree (including itself)
- Update size after every split/merge operation.
- To find k-th element:
  - Compare k with size(left_child) to decide direction
- To count the number of elements of S smaller than x:
  - Compare x with the key of the current node. When going to the right subtree, add size(left_child) + 1 to the answer

Tanks foe washing

Copyright © University of Science, VNUHCM - APCS 2023